

Chronofold: a data structure for versioned text

Victor Grishchenko

N.N. Krasovskii Institute of Mathematics and Mechanics
Ekaterinburg, Russia
gritzko@imm.uran.ru

Mikhail Patrakeev

N.N. Krasovskii Institute of Mathematics and Mechanics
Ekaterinburg, Russia
patrakeev@mail.ru

Abstract

Chronofold is a replicated data structure for versioned text, based on the extended Causal Tree model. Past models of this kind either retrofitted local linear orders to a distributed system (the OT approach) or employed distributed data models locally (the CRDT approach). That caused either extreme fragility in a distributed setting or egregious overheads in local use. Overall, that local/distributed impedance mismatch is cognitively taxing and causes lots of complexity. We solve that by using subjective linear orders locally at each replica, while inter-replica communication uses a distributed model. A separate translation layer insulates local data structures from the distributed environment. We modify the Lamport timestamping scheme to make that translation as trivial as possible. We believe our approach has applications beyond the domain of collaborative editing.

CCS Concepts: • Software and its engineering → Synchronization; Consistency; Software performance.

ACM Reference Format:

Victor Grishchenko and Mikhail Patrakeev. 2020. Chronofold: a data structure for versioned text. In *7th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC '20)*, April 27, 2020, Heraklion, Greece. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3380787.3393680>

1 Introduction

Even without the real-time collaboration, data structures for editable text is a vast field on its own. Plain text storage and transmission is not a challenge for modern computers; “War and Peace” weighs 3MB, on par with a smartphone photograph. Text editing is more demanding, as it needs fast writes and some basic versioning functionality (at least, to support undo/redon). Naive implementations do not suffice; there is an entire class of editable-text data structures,

such as gap buffers [17], piece tables [30], ropes [14] and others. Integrated Development Environments (IDEs) have even more reasons to version the edited text; one of them is asynchronous communication between multiple worker threads or processes. Finally, there are Source Code Management systems (SCM), where texts are versioned and stored long-term. The underlying models of text versioning have plenty of overlap in these three classes of applications.

The classic plain text versioning model sees any document change (a *diff*) as a number of range insertions and deletions. Alternatively, that can be generalized to a number of range replacements (*splices*). The Myers algorithm [31] can calculate a diff from two versions of a text in $O(ND)$ time, where N is the combined size of the texts and D is the size of the changes. Thus, the worst case is $O(N^2)$. That is less of a problem for diff, patch, svn, git, etc, as their unit of change is a line of text. There are much less lines than characters and lines are more unique, so a number of optimizations and heuristics make Myers good enough in all the reasonable cases. If the unit of change is a *character*, Myers is much more of a challenge; e.g. Google’s diff-match-patch [20] library uses timers to provide a good-enough result in acceptable time. Another issue with the diff approach is its non-determinism in case of concurrent changes. To integrate a change, patch relies on its position and *context* (the text around the changed spot). Concurrent changes may garble both, causing mis-application of a patch. Partially, that is solved by heuristics. Still, SCMs require manual merge of changes in any non-trivial cases.

Weave [36] is a classic data structure for text versioning. It was invented in the 1970s and used in SCCS, TeamWare and most recently BitKeeper as the main form of storage and by many other SCMs for merge of concurrent changes. A weave has a reputation of one of the most reinvented data structures in history. It is alternatively known as “interleaved deltas”, “union string”, and under other names. Its key idea is simple: annotate all pieces of a text (*deltas*) with their “birth” and “death” dates, keep the deleted pieces in their place. Then, one pass of such a collection can produce any version of the text, if all the “dead” and “yet-unborn” pieces are filtered out. The top issue with a weave is that it needs to be spliced on every edit (i.e. rewritten in full), very much like a plain string. The original SCCS weave was line-based, but we will use that as a broad term for this kind of a data structure, no matter line- or character-based.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. PaPoC ’20, April 27, 2020, Heraklion, Greece

© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-7524-5/20/04...\$15.00

<https://doi.org/10.1145/3380787.3393680>

Notably, the widely popular git SCM [6] has immutable binary blobs as its primary abstraction, no deltas. Still, it employs delta-based data structures to merge concurrent changes, while its internal storage format is organized around delta compression. It also supports line-based patches and blame maps. Ironically, declaring blobs as its primary abstraction made git use deltas more, not less.

The *Operational Transformation* (OT) model [19] originated from the first experiments with real-time collaborative editors in the 80s. With OT, each single-character edit can be sent out immediately as an *operation* (op). OT needed deterministic merge of changes, despite any concurrent modification. Hence, it relied on positions, not contexts, to apply the changes. Positions are also affected by concurrent edits, so OT iteratively *transforms* the operations to keep them correct. That works reasonably well, except that concurrent modifications create combinatorially complex and highly counter-intuitive effects. For that reason, any practical OT implementation relies on a central server to transform the ops. Despite its somewhat torturous history, OT eventually led to such applications as Google Docs.

In 2006, the dissatisfaction with OT led to a new proposal named *WithOut Operational Transforms* (WOOT) [34]. Its cornerstone change was to assign every character a unique identifier (id). Then, WOOT represents a text as a directed acyclic graph of characters, each one referencing its left and right neighbors at the time of insertion. The order of identifiers resolves ties between concurrent insertions to the same location. Deleted characters get marked with *tombstones*. WOOT ops are immutable and commutative, hence immune to reordering issues. While obviously correct, WOOT was highly impractical due to metadata overheads.

Causal Tree (CT) [22] aimed at improving WOOT, along with Logoot [42], TreeDoc [35], LSEQ [32] and other proposals. In particular, CT reduces per-character metadata to (a) *logical timestamp* (b) timestamp of the preceding character. Logical timestamps [26] are tuples $\langle t, \alpha \rangle$ where t is the logical time value and α is the process id. The lexicographic order of timestamps forms the *arbitrary total order* [27] (ATO) consistent with the cause-effect ordering (“happened-before”). CT employed fixed-width logical timestamps of various kinds, while Logoot and TreeDoc used variable-length identifiers. Once the first CT draft [21] appeared in 2008, it was immediately noted [16] that CT’s inner workings are very reminiscent of a weave. In 2010, the Replicated Growable Array (RGA) [37] algorithm was proposed. In 2016, it was formally proven [12] that RGA and CT use the same algorithm (curiously, the paper uses another term for CT, a Timestamped Insertion Tree). Interestingly, OT-with-tombstones proposals [28, 33] resulted in similar weave-based algorithms.

In 2009, the authors of TreeDoc proposed a broad term for this kind of commutativity/convergence based algorithms:

Conflict-free Replicated Data Types (CRDT) [29, 40]. Although the potential of commutative ops was noticed as far back as 1987 [38], only now the topic had close attention. But, despite both industry and academia making circles around CRDTs, no standard solution emerged yet. The key issue remained the same: metadata overheads and cognitive costs of a distributed data model. So far, industry adoption of CRDTs had the air of a pilot project. By 2013, CT was deployed in the Yandex Live Letters collaborative editor [4, 23] which was phased out several years later. Another CRDT based editor was made in Yandex [2] by 2019. The Atom editor employed CRDT [3] for its collaborative features. Apple Notes is known to sync with CRDTs [9]. Some used simplified ersatz-CRDT models [41]. As recently as in 2019, two high-profile CRDT-based editor projects fell short of the objectives (Google-associated xi [7] and GitHub’s xray [5]). Authors cited data structure complexity as a major impediment.

CRDT overheads and complexity are rooted in the use of per-character ids and the need to map them to positions in the weave and the text. That data has to be stored in addition to the regular editable-text data structures. According to the original article, RGA needs a hash map to store a linked list, one entry per character. A naive CT implementation may store a text as an actual tree of letters, with each letter being a rather complex object. Such implementations are known to exist and they don’t work well. Overall, turning a character into an object brings lots of overheads: pointers, headers, cache misses, garbage collection, etc. Every implementation tried to optimize that, one way or another.

Some tried to work with blocks of characters [11, 15] instead of single characters; e.g. add a copy-pasted fragment as a single op, then split it later if necessary. Another approach was to compress ranges of timestamps [8] which are close in value. The xi editor used a hybrid OT-CRDT approach (“coordinate transforms”) to save on the metadata; predictably, that increased algorithm and data structure complexity. CT survived multiple major revisions addressing the issue of overheads. Most CT implementations used flat data structures: strings, arrays and buffers [8, 13]. In particular, the 2012 JavaScript implementation [22, 23] used a peculiar optimization technique coding timestamps as tuples of characters and using regex scans to avoid keeping a per-symbol hash map. The 2017 RON CT implementation [8] borrowed the iterator heap technique [1] from LSMT databases. It merges i inputs by a single $O(N \log i)$ pass; the inputs might be versions, patches and/or single ops. The technique is perfect for batched server-side operations, not so much for real-time client-side use. So far, optimizations did not lower CRDT overheads to the level of a piece table or at least comparable. That makes CRDTs acceptable for niche uses, but not as a general-purpose data structure for versioned text.

This paper proceeds as follows. Sec. 2 explains the category of subjective linear orders and log timestamps, a logical timestamping scheme. That is the key to the article as it

lets us use linear addressing in a distributed data structure. In Sec. 3, we introduce chronofold, a data structure for versioned text. Further, in Sec. 4 we put chronofold into the wider context of a complex editor or revision control system and explain how it works in lockstep with other data structures. Finally, Sec. 5 concludes with our findings.

2 CT, log time and subjective orders

In distributed systems, events happen “fast” while messages propagate “slowly”. As a result, the perceived order of events is different for different observers. No wonder the seminal paper on distributed systems [27] drew inspiration from relativistic physics; its key concept of *logical time* is dependent on the frame of reference.

The CT model is defined in a way to be independent of any replica’s local perceived operation order (*subjective* order). CT works in terms of a *causal* partial order of operations and a compatible timestamp-based ATO. That makes the model simple and its behavior self-evident. The unfortunate cost is that addressing, data structures, and versioning become non-linear and thus complex.

We found that the inner workings of the system might be greatly simplified if we rely on those linear subjective orders instead of ignoring them. As long as the system produces the same text, we have the best of both worlds: simplicity of linear addressing and resilience of a distributed model. In the Replicated Causal Tree model (RCT) we make subjective orders explicit and consider their properties.

We denote *processes* by variables α, β, γ ; the variables i, j, k, m, n range over the set of natural numbers $\{1, 2, \dots\}$. Processes create and exchange *operations* (ops) identified by *timestamps* (ids). A timestamp is an ordered pair $t = \langle n, \alpha \rangle$ where α is the *author* also denoted by $\text{auth}(t)$, and n is the *author’s index* also denoted by $\text{andx}(t)$. An *op* is a tuple $\langle t, \text{ref}(t), \text{val}(t) \rangle$ where t is its id, $\text{ref}(t)$ is the id of its CT parent, and $\text{val}(t)$ is its value (a character). Each process α keeps a subjectively ordered log of ops it either authored or received from other processes.

Definition 2.1. *Replicated Causal Tree, RCT*, is a tuple $R = \langle T, \text{val}, \text{ref}, \text{log} \rangle$ such that T is a set of timestamps, val is a function with domain T , ref is a function from T to T , and log is a function from the set $\text{proc}(R) := \{\text{auth}(t) : t \in T\}$ to the set of injective sequences in T , which associates to every process $\alpha \in \text{proc}(R)$ the sequence $\text{log}(\alpha) = \langle \alpha^1, \alpha^2, \dots, \alpha^{\text{lh}(\alpha)} \rangle$ of length $\text{lh}(\alpha)$ of timestamps in T with $\alpha^i \neq \alpha^j$ for $i \neq j$, and such that for all $\alpha, \beta \in \text{proc}(R)$ and $i \leq \text{lh}(\alpha)$ the following three axioms are satisfied:

1. If $\langle n, \alpha \rangle \in T$, then $n \leq \text{lh}(\alpha)$ and $\alpha^n = \langle n, \alpha \rangle$.
2. If $\langle n, \alpha \rangle \in T$, then $\text{ref}(\langle n, \alpha \rangle) = \alpha^j$ for some $j \leq n$.
3. If $\alpha^i = \langle m, \beta \rangle$, then $\text{LOG}_m(\beta) \subseteq \text{LOG}_i(\alpha)$, where $\text{LOG}_k(\gamma)$ is the set $\{\gamma^1, \dots, \gamma^k\}$ for $\gamma \in \text{proc}(R)$ and $k \leq \text{lh}(\gamma)$.

Then, the Causal Tree that RCT produces is the directed graph $\langle T, \{\langle t, \text{ref}(t) \rangle : t \in T\} \rangle$. Note that formally the notation α^i is an abbreviation for $(\text{log}(\alpha))_i$, i.e. the i -th term in the sequence $\text{log}(\alpha)$, and this notation can be used only for $\alpha \in \text{proc}(R)$ and $i \leq \text{lh}(\alpha)$. Note also that $\alpha^n = \langle n, \alpha \rangle$ holds only if $\langle n, \alpha \rangle \in T$. Position n in the log of α might be taken by somebody’s else op $\langle m, \beta \rangle$; then, $\langle n, \alpha \rangle \notin T$, see Sec. A.

Notation 2.2. Suppose that $R = \langle T, \text{val}, \text{ref}, \text{log} \rangle$ is an RCT, $t \in T$, and $\alpha \in \text{proc}(R)$. Then

- $\text{ndx}_\alpha(t) := i$ such that $t = \alpha^i$, if $t \in \text{LOG}_{\text{lh}(\alpha)}(\alpha)$;
- $\text{ndx}_\alpha(t) := \infty$, if $t \notin \text{LOG}_{\text{lh}(\alpha)}(\alpha)$.

We call $\text{ndx}_\alpha(t)$ the *index* of the op t in the log of process α . It may be different from its author’s index $\text{andx}(t)$ due to different op orders in these logs (see Fig. 4 for examples).

Remark 2.3. Suppose that $R = \langle T, \text{val}, \text{ref}, \text{log} \rangle$ is an RCT, $t \in T$, $\beta \in \text{proc}(R)$, $k \leq \text{lh}(\beta)$, and $\langle i, \alpha \rangle \in T$. Then:

- $\text{andx}(t) = \text{ndx}_{\text{auth}(t)}(t)$.
- $\text{andx}(\beta^k) \leq k$.
- $\text{andx}(t) \leq \text{ndx}_\beta(t)$ for all $\beta \in \text{proc}(R)$.
- $\text{andx}(t) > \text{andx}(\text{ref}(t))$, if $t \neq \text{ref}(t)$.
- $\text{andx}(\langle i, \alpha \rangle) = i > \text{andx}(\alpha^j)$ for all $j < i$. $\square$

Note that an op’s index in its author’s log is the lowest index it has, in any log. Also, the index of an op in its author’s log is greater than author indices of any preceding ops in the same log, including its CT parent. Even with subjective ordering, these features hold because of causal consistency and the way we defined timestamps. We will rely on these features later.

Definition 2.4 (Causal partial order $\sqsubseteq$). Suppose that $R = \langle T, \text{val}, \text{ref}, \text{log} \rangle$ is an RCT and $s, t \in T$. Then:

- $s \sqsubseteq t$ iff there is a sequence $\langle r_1, \dots, r_n \rangle$ such that
 - $r_1 = s$,
 - $r_n = t$, and
 - $r_i = \text{ref}(r_{i+1})$ or $r_i \in \text{LOG}_{\text{andx}(r_{i+1})}(\text{auth}(r_{i+1}))$ for all $i < n$.
- $s \sqsubset t$ iff $s \sqsubseteq t$ and $s \neq t$.

Remark 2.5 (Consistency of andx and causality). Suppose that $R = \langle T, \text{val}, \text{ref}, \text{log} \rangle$ is an RCT, $s, t \in T$, and $s \sqsubset t$. Then $\text{andx}(s) < \text{andx}(t)$. $\square$

Lemma 2.6. Suppose that R is an RCT, $\alpha \in \text{proc}(R)$, $i \leq \text{lh}(\alpha)$, and $\text{ref}(\alpha^i) = \langle k, \gamma \rangle$. Then:

- $\text{LOG}_k(\gamma) \subseteq \text{LOG}_i(\alpha)$.

That is, $\text{LOG}_{\text{andx}(\text{ref}(\alpha^i))}(\text{auth}(\text{ref}(\alpha^i))) \subseteq \text{LOG}_i(\alpha)$. $\square$

Corollary 2.7 (Causal closedness of $\text{LOG}_i(\alpha)$). Suppose that R is an RCT, $\alpha \in \text{proc}(R)$, and $i \leq \text{lh}(\alpha)$. Then the set $\text{LOG}_i(\alpha)$ is causally closed. That is, if $s \sqsubseteq t$ and $t \in \text{LOG}_i(\alpha)$, then $s \in \text{LOG}_i(\alpha)$. $\square$

Historically, CT used at least five different timestamping schemes (Lamport [27], hybrid [18], abbreviated/char tuple [22], calendar based and others). Given their role in the system, even subtle details had a lot of impact. The scheme defined here is named *log timestamps*. Instead of incrementing the value of the greatest timestamp seen, like the Lamport scheme does, we set it to the op's index in its author's log. The lexicographic ordering of log timestamps is compatible with the causal order (Rem. 2.3). In addition to that, it also provides a lower bound for the op's index in any log. Pragmatically speaking, it is the same number most of the time, as the level of contention between replicas of a text tends to be small. That makes it possible for us to switch from log timestamps to log indices and back, with very little friction.

The importance of this becomes clear when we consider our two uses for timestamps: referencing operations and forming the ATO. Locally, referencing by index is much more convenient. The convenience of using an index for ordering depends on whether it matches the author's index. If $\alpha^n = \langle n, \beta \rangle$ then the index also equals the most lexicographically significant part of the timestamp. If not, it provides an upper bound on that part of the timestamp due to Rem. 2.3. That is enough to determine the ATO in the absolute majority of the cases. So most of the time, an RCT implementation may use indices instead of timestamps.

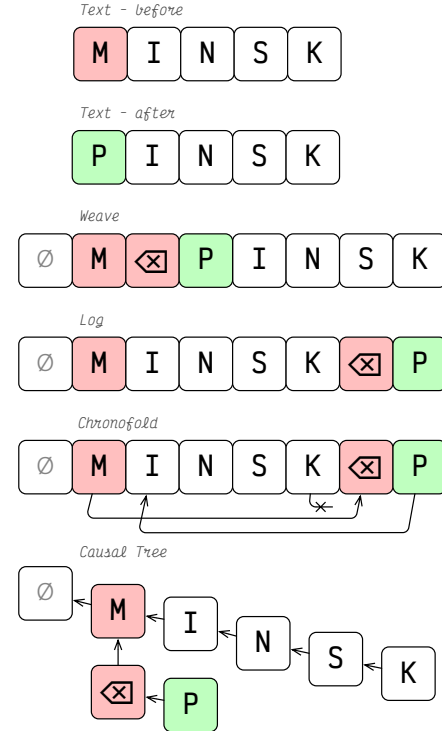
Given that, a process may convert logical timestamps to its log indices once it receives an operation from another process. Then, it proceeds with the indices. When sending operations out, it performs the reverse conversion. This way, we *insulate* data structures from the complexity of a distributed environment.

Another important feature of log indices is their stability. The main source of the famous OT complexity is its reliance on a linear addressing system that is not stable between edits. We avoid that here, to our great advantage, see Sec. 4. Given all of the above, we believe it is as natural to use log timestamps for versioned texts as using quaternions for 3D graphics and modeling. In the next section, we introduce a versioned text data structure that is comparable to plain-text data structures in terms of complexity and overheads.

3 Chronofold

Every data structure for versioned text has its advantages and shortcomings. A plain text string is the most simple and the most extensively used data structure in the world. Unfortunately, a string is edited by splicing; once we insert a character in the middle, we have to rewrite half the text. Hence, text edits are $O(N)$ while comparing (diffing) two versions of a string by the Myers algorithm is $O(ND)$. Weave is the most natural data structure for diffing and merging, but editing a weave also requires splicing. A log is an append-only data structure, hence has $O(1)$ edits. But, recovering a text from a log of edits is not trivial. Notably, log is a *persistent*

Figure 1. Versioned text: “MINSK” corrected to “PINSK”, stored in different data structures ($\emptyset$: root, $\triangleleft$: tombstone).



data structure in the sense that every prefix of a log is also its complete version. Similarly, any postfix of a log is a list of recent changes, which is very convenient for replication and synchronization. Piece tables as used by many text editors have either $O(1)$ or $O(\log N)$ edits and may provide very limited versioning functionality.

So ideally, we want a replicated versioned text data structure that is stored in an array, addressed by indices, needs no splicing, allows access to past versions of the text and merge of remote changes. We achieve that by combining a weave and a log: a *chronofold*¹ is a subjectively ordered log of tuples $\langle \text{val}(\alpha^i), \text{ndx}_\alpha(w(\alpha^i)) \rangle$, $i \leq \text{lh}(\alpha)$, where $w(\alpha^i)$ is the operation following α^i in the weave. So, the second element of the tuple forms a linked list that contains the weave and thus any version of the text. In the C notation, a text chronofold may look like:

```
struct {
    char32_t codepoint; // UTF-32 character
    uint32_t next_ndx;  // weave linked list
} * cfold;
```

By reading a chronofold like a log, we see the history of changes. By reading it along the linked list, we may see any version of the text. A chronofold has the good properties of a log, a weave and a piece table: it is splicing-free, versioned

¹The name of the data structure was decided by a popular vote [10].

and very convenient for synchronization. A chronofold entry takes less space than an op due to the absence of timestamps. We further optimize that in Sec. 4.

The algorithm for merging new ops into a chronofold resembles well-known CT/RGA algorithms [22, 37, 39]. Once process α receives an op $\langle i, \beta \rangle$, it appends an entry to its chronofold. Next, it has to find the op's position in the weave and relink the linked list to include the new op at that position. It locates the new op's CT parent $\text{ref}(\langle i, \beta \rangle) = \langle k, \gamma \rangle = \alpha^j$ at the index j in the local log. Here, $k < i$ and $k \leq j$; most of the time we simply have $j = k$. It inserts the op after its parent, unless it finds *preemptive CT siblings* at that location (those are ops with greater timestamps also having $\langle k, \gamma \rangle$ as their parent). If found, the new op is inserted after preemptive siblings and their CT subtrees.

If explained in RGA terms [37], the CT parent becomes “left cobject” while preemptive siblings become “succeeding nodes” with greater timestamps/vectors. In the terms of the 2010 paper [22], those are “parent” and “unaware siblings”. In plain words, preemptive siblings correspond to concurrent insertions into the same point of the text. It is the trickiest part of CT/RGA, as it is an algorithm-within-an-algorithm with somewhat different properties [25]. Still, it is considered a worthy tradeoff because it keeps other parts simple. In real-world usage, preemptive siblings are very rare.

Note that the chronofold building algorithm uses information that is not included into the chronofold itself. Namely, that is the tree-forming ref relation and the timestamp-to-index mapping $\text{ndx}_\alpha : \langle k, \gamma \rangle \rightarrow j$. It may also need $\text{ndx}_\alpha^{-1} : j \rightarrow \langle k, \gamma \rangle$ for the case of preemptive siblings. Exporting edits to other replicas needs ndx_α^{-1} to produce the timestamps.

Importantly, if we only edit a text locally then a chronofold itself suffices. Namely, as per Remark 2.3, the timestamp of a new locally authored op is greater than other timestamps in the log. That excludes the case of preemptive siblings, so ndx_α^{-1} is not needed. The index of the preceding character should be already known, so ndx_α is not needed either.

Then, the data for ref , ndx_α , ndx_α^{-1} can be kept in a separate data structure thus removing it from the hot code path. From the perspective of a text editor, that makes perfect sense: it merges an op once, then reads many times. This is exactly the insulation layer we mentioned earlier.

The simplest way to store that metadata is to keep a *secondary* log of op timestamps and their ref indices. To implement $\text{ndx}_\alpha^{-1} : j \rightarrow \langle k, \gamma \rangle$ we simply read that log at the index j . To implement $\text{ndx}_\alpha : \langle k, \gamma \rangle \rightarrow j$, we may need to scan it from position k to the end, in the worst case. That costs $O(N)$ and from that perspective we might be tempted to store that mapping in a hash map. That would solve the problem on paper but, as it was described earlier, that may not be a good idea in practice. One way to avoid those worst-case scans is to keep a separate sorted table of index *shifts*. Namely, once $\text{ndx}_\alpha(\langle i, \beta \rangle) - i > T$ for some threshold value

T , make a shift table entry $\text{shft}_\alpha : \langle i, \beta \rangle \rightarrow \text{ndx}_\alpha(\langle i, \beta \rangle) - i$. Having that entry, we will know that for $\langle j, \beta \rangle$ if $j \geq i$ then $\text{ndx}_\alpha(\langle j, \beta \rangle) \geq \text{shft}_\alpha(\langle i, \beta \rangle) + i$. As long as this correction keeps us within T steps from the target, we do not need additional entries for β . This technique is improved in Sec. 4.

With a shift table, ndx has complexity $O(\log N)$, which means $O(\log N)$ insertions, except for one adversarial scenario. Namely, if one op has $O(N)$ CT children which are fed into our replica in the reverse timestamp order. Then, the case of preemptive siblings turns into the bubble sort algorithm: $O(N)$ per op, $O(N^2)$ total. The scenario corresponds to lots of concurrent insertions into the same point in the text. Due to the properties of causal consistency, one author can not send ops out-of-order, see Def. 2.1, Lemma. 2.6. So, this scenario should probably include a Sybil attack [24] too. There is another chronofold-building algorithm that lacks this unfortunate corner case; we have to skip it as it depends on many techniques not explained here.

To illustrate what we achieved here, let us consider two typical versioning operations: recovering a past version and deriving a difference of two versions. Having a CT weave, we would need timestamps and version vectors to filter non-effective ops (“dead”, “yet-unborn”) and produce a version of a text or a difference thereof. Having a chronofold, we may iterate its linked list while ignoring all the ops past certain index. This way, we produce a version or a diff using indices only. (Albeit, this only applies to the versions we observed in our subjective linearization of the history; to work with other linearizations we have to build their respective chronofolds.) As we have mentioned earlier, local editing does not use timestamps either. That should make CRDT overheads acceptable for the use cases of undo/redo, real-time collaborative editing, or full-scale revision control.

But, whether we speak of editors, collaborative editors or revision control systems, there is more than plain text. There is also formatting, highlighting, annotations, versioning. In this regard, log timestamps make the data structure extremely flexible and adaptable, see Sec. 4.

4 Co-structures

All but the most basic editors overlay the text with various kinds of formatting. That might be syntax highlighting, spelling errors, compiler warnings, authorship and versioning info, annotations, etc. Differently from embedded markup (e.g. HTML), overlays are decoupled from the text stream, as they merely *reference* text ranges. Sometimes, the code responsible for such overlays may be computationally expensive, so it runs asynchronously in separate threads, processes or remotely on servers. Sometimes, these overlays are stored separately. Either way, as the text keeps changing, the referenced ranges become slightly off. Effectively, some editors have to run miniature OT engines to correct for that effect.

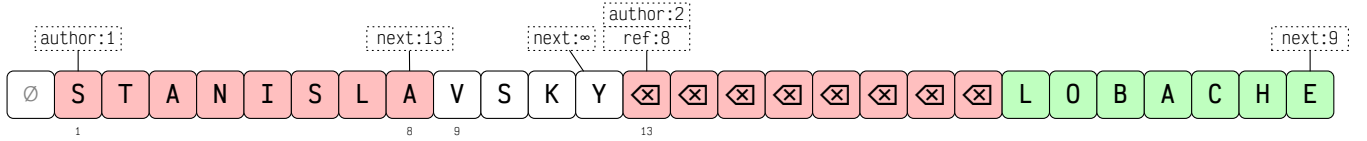


Figure 2. Chronofold and co-structures: "STANISLAVSKY" written by author 1 corrected to "LOBACHEVSKY" by author 2. Co-structures store timestamps (author, shift), the weave (next), and the tree-forming relation ref. Most values are implicit.

Fortunately, log indices create a stable addressing system for the edited text. As long as we stay with the same replica and same linearization, the indices are not affected by edits. That lets us build *co-structures*, overlay data structures linked to the text through log indices. Co-structures reference text ranges, but instead of text positions they use log indices, so no correction needed. This is an improvement over past CT editor engines that used logical timestamps to denote such ranges. Again, we evaded the use of distributed primitives.

As a simple example, we may track a binary attribute by keeping a bitmap (e.g. whether a letter is bold). For richer attributes we may use a vector, etc. Although, keeping track of individual characters may not be the most convenient approach. In case we need to reference character ranges, one possible data structure is a *range map*. Namely, we divide a chronofold into a number of semi-intervals $[a_i, b_i)$ so $a_i = b_{i-1}$, $i \neq 0$. Each interval has uniform formatting f_i . That formatting we keep in a sorted map $a_i \rightarrow f_i$. When iterating the chronofold in the weave order, we check the range map for formatting changes.

As text editors tend to operate in (row,column) coordinates, we may dedicate another co-structure to that purpose. Namely, a Table of Contents (ToC) listing log indices of all the effective newlines of the text in their weave order. Having that, we can start iteration from an arbitrary line's beginning. This way, we may avoid storing the plain text as a separate structure. Instead, we may produce any line of the text on-demand by scanning the respective piece of the weave. Again, co-structures make a chronofold extremely flexible.

Note that a slightly out-of-date co-structure can still be applied to the chronofold if that makes sense. As co-structures are decoupled from the chronofold, they can be (de)activated, (re)stored, (re)built and/or updated, all independently from the editing process. The only limitation is that the subjective order must stay the same.

Interestingly, this co-structure technique may serve to optimize the chronofold itself. In part, we already did that in the Sec. 3 by offloading metadata to a secondary log. As a next step, we may offload `next_ndx` pointers to a co-structure of their own. A typical versioned text consists of *spans* of sequentially typed characters: words, sentences, deltas. Simultaneous typing in a real-time collaborative editor may produce messier patterns. But, based on our experience with deployed systems, that is a rare exception, not the rule. In a typical chronofold, most of `next_ndx` pointers would be

equal to $i + 1$. Instead of spending memory for every such value, we may offload them to a separate sparse array, where only non-trivial pointers are mentioned. In the resulting implementation, a *thinned* chronofold is simply a log of UTF-32 codepoints. As yet another optimization, we may notice that non-BMP codepoints are very rare. If so, we may reduce the core chronofold to a UTF-16 string where all non-BMP codepoints are marked with a special value and stored in a yet another sparse array.

Let's return to the secondary log carrying op timestamps and ref indices (Sec. 3). Author's indices tend to match our local log indices in practice. Even if not, spans will have the same index *shift* due to concurrent edits present in our log before the span. So, instead of individual timestamps we may store timestamp ranges in two range maps (authors and shifts respectively), thus avoiding per-character metadata. In this case, ndx_{α}^{-1} takes $O(\log N)$ cycles as it only needs two range map lookups. ndx_{α} is formally $O(N)$ as it may potentially need to scan a range map to find an op that was shifted from beginning to end. Optimizing this case is possible, but hardly worth it in non-adversarial scenarios, as we need ndx_{α} for head-of-span insertions only.

As the final optimization in this paper, we will use the fact that all the co-structures are addressed by the log index. That means, it is possible to put several of them into the same container, to amortize costs. Fig. 2 shows a chronofold with its secondary log and weave pointer co-structures. These can be stored in the same sorted map, assuming the integer key has two flag bits to differentiate between co-structures.

Our span-friendly technique can be even more effective if we discard the history once it becomes irrelevant. In `git` terms, that is called a *rebase*, a mandatory procedure in larger projects. As an extreme case, if a history of a document is discarded entirely, then the text is represented as a single sequential insertion, with no tombstones. In such a case, a text, a weave and a chronofold become the same sequence. The `next_ndx`, author and shift co-structures will have one entry each. If we start editing such a rebased text, a chronofold would look very much like a piece table: an initial snapshot and a separate log for the new edits.

This way or another, if the cost of co-structures is sufficiently amortized, a chronofold's footprint becomes much closer to that of a piece table, or a plain non-versioned UTF-16 string, as used in Java, JavaScript, etc. That makes it useful as a general-purpose data structure for versioned text.

5 Conclusion

As a data structure, chronofold addresses the shortcomings of weave-based CRDTs. It is a simple array-based data structure with $O(1)$ inserts that might work faster than a plain string in many cases. It works like a piece table for editing, like a log for replication, and like a weave for versioning. The authors are looking forward to see chronofold applications in the domains of revision control systems, collaborative software and development environments.

The greatest surprise to the authors though is that linear addressing is applicable to a partially ordered system. The concepts of log timestamps and subjective linear orders mitigate the cognitive and computational costs of a distributed data model. That may potentially find applications beyond the domain of text versioning.

6 Acknowledgements

The work was supported by the Protocol Labs RFP program. Authors thank Anton Lipin, Ilya Sukhopluev, Maxim Shafirov, and Yuriy Syrovetskiy for sharing their reflections and feedback.

References

- [1] Cassandra - MergeIterator. <https://github.com/apache/cassandra/blob/trunk/src/java/org/apache/cassandra/utils/MergeIterator.java>.
- [2] Codefest'19 talk, "A CRDT text editor goes to production" (Russian). <https://2019.codefest.ru/lecture/1501>.
- [3] Collaborate in real time in Atom, <https://teletype.atom.io/>.
- [4] A CRDT-based collaborative editor engine of letters.yandex.ru (2012, historical) <https://github.com/gritzko/citrea-model>.
- [5] An experimental next-generation Electron-based text editor. <https://github.com/atom-archive/xray>.
- [6] Global information tracker, <https://git-scm.com/>.
- [7] A modern editor with a backend written in Rust. <https://xi-editor.io/>.
- [8] Replicated Object Notation 2.0.1, <https://github.com/gritzko/ron>.
- [9] Revisiting Apple Notes (3): Embedded tables, <https://ciofecaforensics.com/2020/01/14/apple-notes-revisited-embedded-tables/>.
- [10] A Twitter vote <https://twitter.com/gritzko/status/1156782170041700352>.
- [11] Luc André, Stéphane Martin, Gérald Oster, and Claudia-Lavinia Ignat. Supporting adaptable granularity of changes for massive-scale collaborative editing. In *9th IEEE International Conference on Collaborative Computing: Networking, Applications and Worksharing*, pages 50–59. IEEE, 2013.
- [12] Hagit Attiya, Sebastian Burckhardt, Alexey Gotsman, Adam Morrison, Hongseok Yang, and Marek Zawirski. Specification and complexity of collaborative text editing. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing*, pages 259–268, 2016.
- [13] Alexei Baboulevich. Data laced with history, <http://archagon.net/blog/2018/03/24/data-laced-with-history/#causal-trees-in-depth>.
- [14] Hans-J. Boehm, Russ Atkinson, and Michael Plass. Ropes: An alternative to strings. *Software: Practice and Experience*, 25(12):1315–1330, 1995.
- [15] Loïck Briot, Pascal Urso, and Marc Shapiro. High responsiveness for group editing crdts. In *Proceedings of the 19th International Conference on Supporting Group Work*, pages 51–60, 2016.
- [16] Bram Cohen. LiveJournal. <https://bramcohen.livejournal.com/49535.html?thread=765311#t765311>.
- [17] Charles Crowley. Data structures for text sequences <https://www.cs.unm.edu/~crowley/papers/sds.pdf>.
- [18] Murat Demirbas, Marcelo Leone, Bharadwaj Avva, Deepak Madeppa, and Sandeep Kulkarni. Logical physical clocks and consistent snapshots in globally distributed databases. 2014.
- [19] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. *ACM SIGMOD Record*, 1989.
- [20] Neil Fraser. The diff match patch library <https://github.com/google/diff-match-patch>.
- [21] Victor Grishchenko. Causal trees: towards real-time read-write hyper-text. <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.627.5286>.
- [22] Victor Grishchenko. Deep hypertext with embedded revision control implemented in regular expressions. *Proceedings of the 6th International Symposium on Wikis and Open Collaboration - WikiSym 10*, 2010.
- [23] Victor Grishchenko. Citrea and Swarm: Partially ordered op logs in the browser. In *1st Workshop on Principles and Practice of Eventual Consistency (PaPEC)*. <https://doi.org/10.1145/2596631.2596641>, 2014.
- [24] Douceur J.R. The Sybil attack. in: Druschel p., Kaashoek f., Rowstron a. (eds) Peer-to-Peer Systems. IPTPS 2002. Lecture Notes in Computer Science, vol 2429. 2002.
- [25] Martin Kleppmann, Victor BF Gomes, Dominic P Mulligan, and Alastair R Beresford. Interleaving anomalies in collaborative text editors. In *Proceedings of the 6th Workshop on Principles and Practice of Consistency for Distributed Data*, pages 1–7, 2019.
- [26] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. In *Concurrency: the Works of Leslie Lamport*, pages 179–196. 2019.
- [27] Leslie Lamport and Clocks Time. the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
- [28] Raph Levien. Towards a unified theory of Operational Transformation and CRDT <https://medium.com/@raphlinus/towards-a-unified-theory-of-operational-transformation-and-crdt-70485876f72f>.
- [29] M Mihai Letia, N Pregoica, and M Shapiro. CRDTs: Consistency without concurrency control. *RR-6956*, INRIA, 2009.
- [30] J Strother Moore. "My" best ideas. <https://www.cs.utexas.edu/users/moore/best-ideas/structure-sharing/text-editing.html>.
- [31] Eugene W Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1(1-4):251–266, 1986.
- [32] Brice Nédelec, Pascal Molli, Achour Mostefaoui, and Emmanuel Desmontils. Lseq: An adaptive structure for sequences in distributed collaborative editing. In *Proceedings of the 2013 ACM Symposium on Document Engineering, DocEng '13*, page 37–46, New York, NY, USA, 2013. Association for Computing Machinery.
- [33] G. Oster, P. Molli, P. Urso, and A. Imine. Tombstone transformation functions for ensuring consistency in collaborative editing systems. In *2006 International Conference on Collaborative Computing: Networking, Applications and Worksharing*, pages 1–10, Nov 2006.
- [34] Gérald Oster, Pascal Urso, Pascal Molli, Hala Skaf-Molli, and Abdessamad Imine. Optimistic Replication for Massive Collaborative Editing. Research Report RR-5719, INRIA, 2005.
- [35] Nuno Pregoica, Joan Manuel Marques, Marc Shapiro, and Mihai Letia. A commutative replicated data type for cooperative editing. In *Proceedings of the 2009 29th IEEE International Conference on Distributed Computing Systems, ICDCS '09*, pages 395–403, Washington, DC, USA, 2009. IEEE Computer Society.
- [36] Marc J Rochkind. The source code control system. *IEEE transactions on Software Engineering*, (4):364–370, 1975.
- [37] Hyun-Gul Roh, Myeongjae Jeon, Jin-Soo Kim, and Joonwon Lee. Replicated abstract data types: Building blocks for collaborative applications. *Journal of Parallel and Distributed Computing*, 71(3):354 – 368, 2011.
- [38] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Comput. Surv.*, 22(4):299–319,

December 1990.

- [39] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A comprehensive study of convergent and commutative replicated data types. 2011.
- [40] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Symposium on Self-Stabilizing Systems*, pages 386–400. Springer, 2011.
- [41] Evan Wallace. How Figma’s multiplayer technology works. <https://www.figma.com/blog/how-figmas-multiplayer-technology-works/>.
- [42] Stephane Weiss, Pascal Urso, and Pascal Molli. Logoot-undo: Distributed collaborative editing system on p2p networks. *IEEE Transactions on Parallel and Distributed Systems*, 21(8):1162–1174, 2010.

A RCT Example

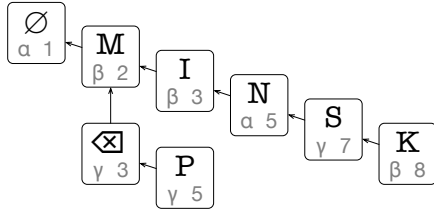


Figure 3. Three laborous editors struggle to spell “PINSK”

In the above RCT example, three processes are named Alice (α), Bob (β), and George (γ). Alice produced ops

$$\langle \langle 1, \alpha \rangle, \langle 1, \alpha \rangle, \emptyset \rangle \text{ and } \langle \langle 5, \alpha \rangle, \langle 3, \beta \rangle, N \rangle;$$

Bob produced ops

$$\langle \langle 2, \beta \rangle, \langle 1, \alpha \rangle, M \rangle, \langle \langle 3, \beta \rangle, \langle 2, \beta \rangle, I \rangle, \text{ and } \langle \langle 8, \beta \rangle, \langle 7, \gamma \rangle, K \rangle;$$

and finally George produced ops

$$\langle \langle 3, \gamma \rangle, \langle 2, \beta \rangle, \triangleleft \rangle, \langle \langle 5, \gamma \rangle, \langle 3, \gamma \rangle, P \rangle, \text{ and } \langle \langle 7, \gamma \rangle, \langle 5, \alpha \rangle, S \rangle.$$

Then we have the following Replicated Causal Tree $R = \langle T, \text{val}, \text{ref}, \text{log} \rangle$. The set of timestamps T is

$$\{ \langle 1, \alpha \rangle, \langle 5, \alpha \rangle, \langle 2, \beta \rangle, \langle 3, \beta \rangle, \langle 8, \beta \rangle, \langle 3, \gamma \rangle, \langle 5, \gamma \rangle, \langle 7, \gamma \rangle \};$$

the function val has the following values on the above timestamps (written in the same order):

$$\emptyset, N, M, I, K, \triangleleft, P, S;$$

the function ref , that defines the places of application of the above characters, has the following values:

$$\langle 1, \alpha \rangle, \langle 3, \beta \rangle, \langle 1, \alpha \rangle, \langle 2, \beta \rangle, \langle 7, \gamma \rangle, \langle 2, \beta \rangle, \langle 3, \gamma \rangle, \langle 5, \alpha \rangle.$$

The set of processes of our RCT is $\text{proc}(R) = \{ \alpha, \beta, \gamma \}$. The logs of these processes depend on the order in which they received ops from each other. For example, the logs could be the following:

$$\text{log}(\alpha) = \langle \alpha^1, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6, \alpha^7, \alpha^8 \rangle =$$

$$= \langle \langle 1, \alpha \rangle, \langle 2, \beta \rangle, \langle 3, \gamma \rangle, \langle 3, \beta \rangle, \langle 5, \alpha \rangle, \langle 5, \gamma \rangle, \langle 7, \gamma \rangle, \langle 8, \beta \rangle \rangle;$$

$$\text{log}(\beta) = \langle \langle 1, \alpha \rangle, \langle 2, \beta \rangle, \langle 3, \beta \rangle, \langle 3, \gamma \rangle, \langle 5, \gamma \rangle, \langle 5, \alpha \rangle, \langle 7, \gamma \rangle, \langle 8, \beta \rangle \rangle;$$

$$\text{log}(\gamma) = \langle \langle 1, \alpha \rangle, \langle 2, \beta \rangle, \langle 3, \gamma \rangle, \langle 3, \beta \rangle, \langle 5, \gamma \rangle, \langle 5, \alpha \rangle, \langle 7, \gamma \rangle \rangle.$$

In this example $\text{lh}(\alpha) = \text{lh}(\beta) = 8$ (because both processes α and β have received all eight ops) and $\text{lh}(\gamma) = 7$ (since the process γ has not received the op with id $\langle 8, \beta \rangle$).

The fourth timestamp in the log of process α is $\alpha^4 = \langle 3, \beta \rangle$. The author of this timestamp is β : we have $\text{auth}(\alpha^4) = \text{auth}(\langle 3, \beta \rangle) = \beta$. The author’s index of this timestamp is 3: we have $\text{andx}(\alpha^4) = \text{andx}(\langle 3, \beta \rangle) = 3$. Also we have $\text{ref}(\alpha^4) = \langle 2, \beta \rangle$ and $\text{val}(\alpha^4) = I$. From this information we can restore the op that this timestamp identifies: the op with id α^4 is $\langle \langle 3, \beta \rangle, \langle 2, \beta \rangle, I \rangle$.

Also we have

$$\text{LOG}_4(\alpha) := \{ \alpha^1, \alpha^2, \alpha^3, \alpha^4 \} =$$

$$\{ \langle 1, \alpha \rangle, \langle 2, \beta \rangle, \langle 3, \gamma \rangle, \langle 3, \beta \rangle \} = \{ \langle 3, \beta \rangle, \langle 2, \beta \rangle, \langle 1, \alpha \rangle, \langle 3, \gamma \rangle \}$$

— the order is irrelevant here because $\text{LOG}_4(\alpha)$ is a *set*, not a sequence (unlike $\text{log}(\alpha)$).

It is not hard to verify that the axioms (1)–(3) are satisfied in the above RCT.

Further on, we have

$$\text{ndx}_\alpha(\langle 3, \gamma \rangle) = 3 \quad \text{because } \langle 3, \gamma \rangle = \alpha^3;$$

$$\text{ndx}_\beta(\langle 3, \gamma \rangle) = 4 \quad \text{because } \langle 3, \gamma \rangle = \beta^4;$$

$$\text{ndx}_\gamma(\langle 3, \gamma \rangle) = 3 \quad \text{because } \langle 3, \gamma \rangle = \gamma^3.$$

Note that $\text{andx}(\langle 3, \gamma \rangle) = 3$ and $\text{auth}(\langle 3, \gamma \rangle) = \gamma$. Also we have $\text{ndx}_\gamma(\langle 8, \beta \rangle) = \infty$ because there is no i such that $\langle 8, \beta \rangle = \gamma^i$.

B On abiding the rules

One may wonder, how a process may ensure it abides the axioms in 2.1. Even more interestingly, how it can ensure other abide the rules too?

The axiom (3) is satisfied automatically if each process relays the ops in its subjective order. The axiom (2) is satisfied if every process only creates op that reference previous ops in the log. To satisfy the axiom (1) each process must properly assign the author’s index to each new op.

On the other end of the connection, how a process may ensure the received op does not violate the axioms? To enforce (1) a process α must keep track of the log indices ndx_β of the process β it synchronizes with. This way, it may detect out-of-order indices $\text{andx}(t) < \text{ndx}_\beta(t)$ and improperly assigned indices $\text{auth}(t) = \beta$, $\text{andx}(t) \neq \text{ndx}_\beta(t)$. To satisfy (2), it should abort synchronization if a newly received op references an op not in the log.

Enforcing (3) transitively as well as op tampering prevention can be achieved by employing Merkle structures, which is well out of the scope of this paper.

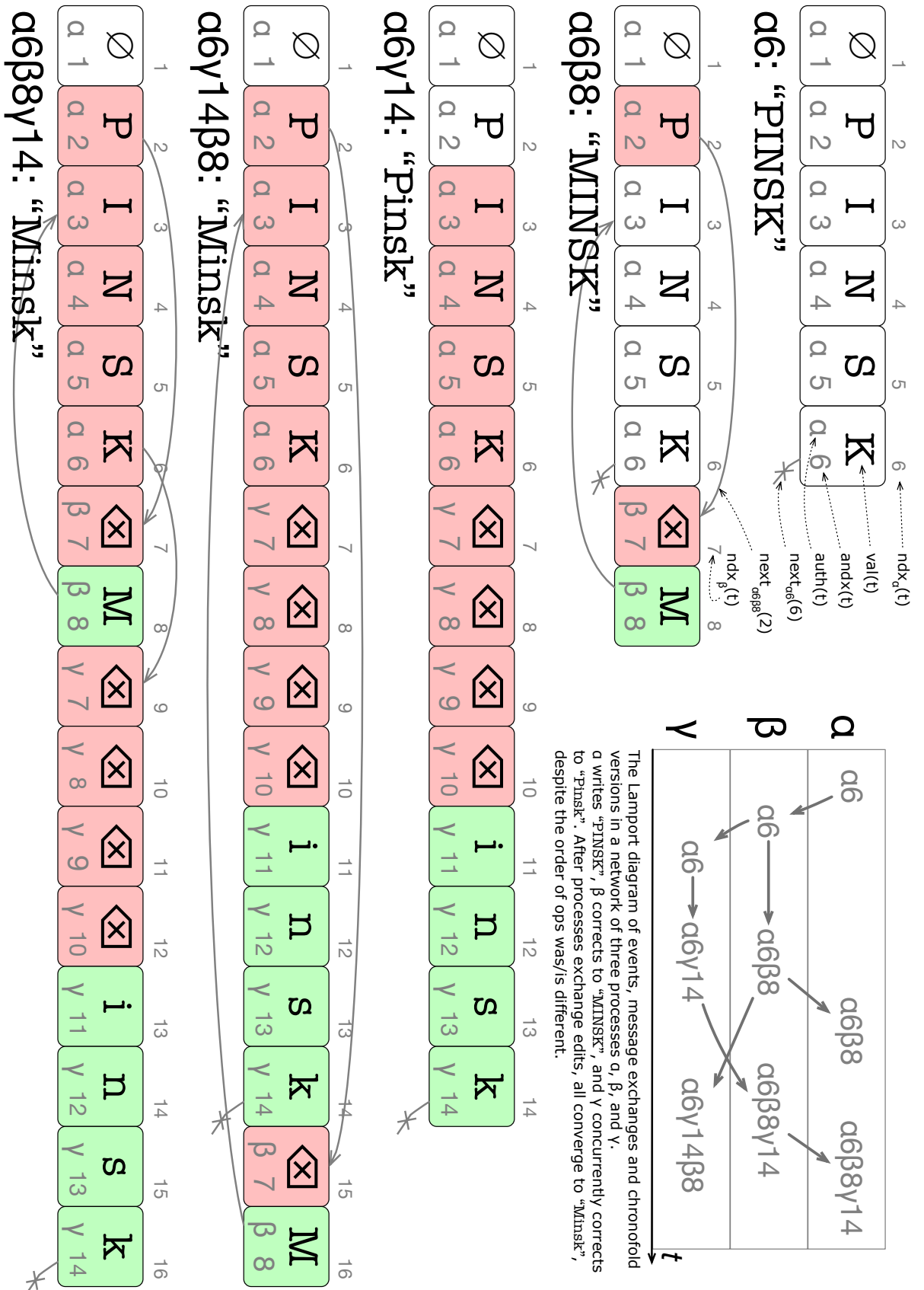


Figure 4. Chronofold Example: three processes, concurrent edits.